



Tienda online de Grafiti - UrbanInk

Santiago Ramírez Pérez

David Santiago Cuestas

FIDI – Facultad de ingeniería, diseño e innovación

Adriana Margarita Meléndez Olivera

Paradigmas de Programación

Tabla de contenido

INTRODUCCIÓN	3
JUSTIFICACIÓN	4
OBJETIVO GENERAL.....	5
OBJETIVOS ESPECÍFICOS	5
MARCO TEÓRICO.....	5
1.1. PROGRAMACIÓN ORIENTADA A OBJETOS (POO)	5
1.2. CLASES Y OBJETOS EN LA POO	6
1.3. PILARES DE LA POO	6
1.4. ENCAPSULAMIENTO	7
1.5. HERENCIA	7
1.6. POLIMORFISMO.....	7
1.7. ABSTRACCIÓN.....	8
2. DIAGRAMAS UML	8
2.1. DIAGRAMA DE CLASES	8
2.2. DIAGRAMA DE CASOS DE USO	9
2.3. DIAGRAMA DE ACTIVIDADES	9
2.4. HISTORIAS DE USUARIO	9
3. MODULARIDAD DE SOFTWARE.....	9
4. TIENDAS ONLINE.....	10
4.1. FUNCIONAMIENTO DE TIENDAS ONLINE	10
DIAGRAMAS UML	11
DIAGRAMA DE CLASES	11
CASOS DE USO	11
HISTORIAS DE CASOS DE USO	12
1. ACCESO AL SISTEMA.....	12
2. SALIDA DEL SISTEMA	12
3. ALTA DE PRODUCTO.....	13
4. BAJA DE PRODUCTO	13
5. GESTIÓN DE INVENTARIO	13
6. FINALIZAR COMPRA	14
7. AÑADIR AL CARRITO	14
8. QUITAR DEL CARRITO	14
9. LIMPIAR CARRITO	15
10. VALIDACIÓN DE PEDIDO	15
11. GESTIÓN DEL PAGO	15
12. EMISIÓN DE FACTURA	16

13. DESPACHO DE PEDIDO	16
CASOS DE USO	16
DIAGRAMA 1:	16
DIAGRAMA 2:	17
DIAGRAMA 3:	17
DIAGRAMA 4:	18
DIAGRAMAS DE ACTIVIDADES	18
DIAGRAMA 1:	18
DIAGRAMA 2:	19
DIAGRAMA 3:	19
DIAGRAMA 4:	20
CONCLUSIÓN	20
REFERENCIAS.....	21

Resumen

UrbanInk es un proyecto académico desarrollado para la asignatura de Paradigmas de Programación del Politécnico Grancolombiano, que consiste en el modelado de una tienda en línea especializada en productos para grafiti y muralismo. El sistema aplica los cuatro pilares de la Programación Orientada a Objetos: encapsulamiento, herencia, polimorfismo y abstracción sobre entidades principales como Usuario, Administrador, Cliente, Producto (con subclases Aerosol, Boquilla y Marcador), Carrito, Pedido, Pago y Envío. Para documentar y visualizar el sistema se utilizaron diagramas UML de clases, casos de uso y actividades, definiendo 13 casos de uso que cubren desde el inicio de sesión y la gestión de inventario por parte del administrador, hasta el proceso de compra, pago y despacho del pedido por parte del cliente. La organización modular del proyecto facilita su comprensión, mantenimiento y posible expansión, y el trabajo en su conjunto logra trasladar los conceptos teóricos del semestre a un contexto funcional cercano a un entorno real de desarrollo de software.

Abstract

UrbanInk is an academic project built by Santiago Ramírez Pérez and David Santiago Cuestas for their Programming Paradigms course at Politécnico Grancolombiano, centered around the design of an online store that sells graffiti and mural art supplies. The system is structured around the four core principles of Object-Oriented Programming, encapsulation, inheritance, polymorphism, and abstraction, applied across the main entities of the system: User, Administrator, Client, Product (broken down into Aerosol, Nozzle, and Marker subclasses), Cart, Order, Payment, and Shipping. UML class, use case, and activity diagrams were used throughout to map out the system's structure and behavior, covering 13 use cases that range from login and inventory management on the admin side, to browsing, checkout, payment processing, and order dispatch on the customer side. The project follows a modular design that keeps the codebase clean and easy to maintain or scale, and overall does a solid job of taking the semester's theoretical concepts and grounding them in something that actually resembles how real-world software gets built.

Introducción

El proyecto consiste en la simulación de una tienda online especializada en la venta de productos para el grafiti y muralismo, cuyo objetivo principal es gestionar clientes, productos, cantidades disponibles y procesos básicos de compra. De esta manera, se busca representar un caso práctico similar a un entorno real de comercio electrónico.

El sistema tiene como propósito que los estudiantes comprendan cómo se aplican los conocimientos desarrollados en clase, especialmente los cuatro pilares de la programación orientada a objetos. En este sentido, se implementa el encapsulamiento mediante el uso de atributos privados y métodos de acceso, la herencia para la reutilización de código entre clases, el polimorfismo a través de la sobreescritura de métodos y la abstracción mediante la definición de estructuras base.

A nivel técnico, el proyecto se encuentra organizado de manera modular, separando las entidades del sistema y la lógica de funcionamiento, lo cual facilita la comprensión, el mantenimiento y la escalabilidad del código. Para la gestión de datos como clientes y productos.

Finalmente, este proyecto no solo muestra la manera en la cual funciona la programación orientada a objetos, sino que también permite desarrollar habilidades prácticas en el diseño e implementación de software, generando una base sólida para futuros proyectos en contextos reales.

Justificación

Este proyecto se justifica por la necesidad de fortalecer el desarrollo académico mediante la aplicación práctica de las aptitudes adquiridas a lo largo del semestre, no únicamente en la asignatura de paradigmas de la programación, sino también en otras áreas relacionadas. A través de este trabajo, se integran distintos conocimientos para el desarrollo de una tienda online, permitiendo llevar la teoría a un contexto funcional.

Se busca trasladar estos conocimientos a un entorno más cercano a situaciones reales de trabajo, donde el estudiante pueda enfrentarse a la estructuración de un sistema y la resolución de problemas mediante programación.

La importancia de este proyecto radica en comprender cómo se estructuran y organizan sistemas basados en objetos, facilitando la implementación de los cuatro pilares de la programación orientada a objetos: encapsulamiento, herencia, polimorfismo, abstracción, excepciones e interfaces. Esto permite no solo entender la teoría, sino aplicarla en un caso práctico.

Asimismo, el desarrollo de una tienda online especializada en productos para grafiti y muralismo permite simular procesos reales como la gestión de clientes, productos e inventario, fortaleciendo habilidades en lógica de programación y diseño de software.

Finalmente, este proyecto resulta viable ya que se cuenta con los conocimientos básicos adquiridos durante el semestre y las herramientas necesarias para su implementación, lo que permite cumplir con los objetivos planteados dentro del contexto académico y aportar a la formación integral del estudiante.

Objetivo General

Desarrollar el modelado de un sistema de tienda en línea especializada en productos de grafiti, mediante la aplicación de los principios de la programación orientada a objetos (POO), con el fin de representar de manera estructurada los procesos de gestión de usuarios, productos y compras, proponiendo una organización modular del sistema basada en los conocimientos adquiridos durante el semestre y orientada a su aplicación en contextos reales.

Objetivos Específicos

- Identificar y definir las entidades principales del sistema (Cliente, Producto, Pedido, Pago y Envío), a partir del análisis del funcionamiento de una tienda en línea.
- Diseñar un modelo orientado a objetos mediante un diagrama de clases UML que represente las relaciones, atributos y métodos del sistema.
- Aplicar los principios de la programación orientada a objetos, incluyendo abstracción, encapsulamiento, herencia y modularidad, en la estructuración del sistema para facilitar su comprensión, mantenimiento y posible ampliación.

Marco teórico

1.1. Programación orientada a objetos (POO)

La programación orientada a objetos (POO) es un enfoque de desarrollo de software que estructura el código a partir de objetos, entendidos como representaciones de entidades que poseen atributos y comportamientos específicos. Este paradigma permite abordar sistemas complejos de forma más

organizada y comprensible, ya que se asemeja a situaciones del mundo real, lo que a su vez favorece la reutilización del código y facilita su mantenimiento. De acuerdo con Grady Booch, uno de los principales referentes en este campo, la POO se fundamenta en principios como el encapsulamiento, la herencia, el polimorfismo y la abstracción, los cuales contribuyen a la creación de sistemas más modulares y escalables. De igual manera, autores como Erich Gamma señalan que este paradigma optimiza la estructura del software al fomentar la separación de responsabilidades y el uso de componentes reutilizables en el desarrollo de aplicaciones (Booch, 2007; Gamma et al., 1994).

1.2. Clases y objetos en la POO

Dentro de la programación orientada a objetos, las clases y los objetos son la base para desarrollar sistemas, ya que permiten representar elementos del mundo real dentro del software. Gracias a estos conceptos, se facilita la organización del código, su reutilización y su comprensión. Además, permiten modelar sistemas de forma estructurada, definiendo relaciones y comportamientos claros entre los distintos componentes del programa (Schildt, 2017; Booch, 2007).

1.3. Pilares de la POO

Los principios de la programación orientada a objetos son fundamentales para el diseño y construcción de sistemas organizados y fáciles de mantener, ya que permiten estructurar el software mediante objetos que combinan tanto datos como comportamientos. Entre estos principios se encuentran el encapsulamiento, la herencia, el polimorfismo y la abstracción, los cuales hacen posible mejorar la reutilización del código, así como la modularidad y la

capacidad de crecimiento de las aplicaciones. Por esta razón, se consideran elementos clave dentro del desarrollo de software actual (Sommerville, 2011; Booch, 2007).

1.4. Encapsulamiento

El encapsulamiento es uno de los principios clave de la programación orientada a objetos y se basa en limitar el acceso directo a los datos que pertenecen a un objeto, de manera que solo puedan ser modificados o consultados a través de métodos previamente definidos. Esto se implementa utilizando modificadores de acceso como público y privado, con el propósito de proteger la información y asegurar que su uso dentro del sistema sea controlado y adecuado (Schildt, 2017).

1.5. Herencia

La herencia es un mecanismo de la programación orientada a objetos que permite definir nuevas clases tomando como base otras ya existentes, aprovechando tanto sus atributos como sus métodos. Gracias a esto, se logra una mejor organización del código y se reduce la duplicación, ya que se pueden establecer relaciones jerárquicas entre clases y reutilizar componentes dentro del sistema de manera más eficiente (Booch, 2007).

1.6. Polimorfismo

El polimorfismo es un principio que permite que un mismo método se comporte de distintas maneras según el objeto que lo invoque. Esto se consigue a través de la sobrecarga o sobreescritura de métodos, lo cual

aporta mayor flexibilidad al sistema y facilita el manejo uniforme de objetos que pertenecen a diferentes clases (Gamma et al., 1994).

1.7. Abstracción

La abstracción es un principio de la programación orientada a objetos que se enfoca en mostrar únicamente las características más importantes de un objeto, dejando de lado los detalles que no son necesarios. De esta manera, se simplifica su uso y se logra representar sistemas complejos de forma más clara, centrando la atención en lo que el objeto hace y no en cómo lo hace, lo que facilita tanto el diseño como la comprensión del sistema (Sommerville, 2011).

2. Diagramas UML

El Lenguaje Unificado de Modelado (UML) es una herramienta utilizada en la ingeniería de software que permite representar de forma visual tanto la estructura como el comportamiento de un sistema. Su uso facilita el modelado, diseño y documentación de aplicaciones antes de llevarlas a la implementación, lo que ayuda a comprender mejor los procesos y mejora la comunicación entre los desarrolladores. UML incluye distintos tipos de diagramas que permiten analizar tanto la parte estructural como las interacciones dentro de un sistema (Sommerville, 2011; Booch, 2007).

2.1. Diagrama de clases

El diagrama de clases es uno de los más relevantes dentro de UML, ya que se utiliza para mostrar la estructura estática de un sistema a través de clases, sus atributos, métodos y las relaciones que existen entre ellas. Este tipo de diagrama resulta clave en la programación

orientada a objetos, porque sirve como guía para la implementación del sistema y permite entender cómo están organizados los componentes principales del software (Booch, 2007).

2.2. Diagrama de casos de uso

El diagrama de casos de uso permite representar las interacciones entre los usuarios (actores) y el sistema, mostrando las funcionalidades que este ofrece. Este tipo de diagrama es útil para identificar los requerimientos del sistema desde el punto de vista del usuario, facilitando la comprensión de lo que el sistema debe hacer (Sommerville, 2011).

2.3. Diagrama de actividades

El diagrama de actividades describe el flujo de procesos dentro del sistema, representando de manera gráfica las acciones, decisiones y secuencias de ejecución. Este diagrama es útil para modelar procesos dinámicos y entender cómo se desarrollan las operaciones dentro de la aplicación (Booch, 2007).

2.4. Historias de usuario

La historia de usuario es una técnica que se utiliza para describir de forma clara y sencilla las necesidades que tiene un usuario dentro de un sistema. Por lo general, se plantea indicando qué desea hacer el usuario y con qué propósito, lo que permite enfocar el desarrollo del software en la solución de problemas reales. Este enfoque ayuda a identificar mejor los requerimientos y favorece la comunicación entre el desarrollador y el usuario final (Sommerville, 2011).

3. Modularidad de software

El modularidad del software es un principio de diseño que se basa en dividir un sistema en partes más pequeñas e independientes, conocidas como módulos, donde cada uno cumple una función específica. Este enfoque permite organizar el código de forma más clara, lo

que facilita su comprensión, mantenimiento y reutilización. Además, ayuda a disminuir la complejidad del sistema, ya que cada módulo puede desarrollarse y analizarse por separado, haciendo más eficiente el proceso de desarrollo (Sommerville, 2011).

4. Tiendas online

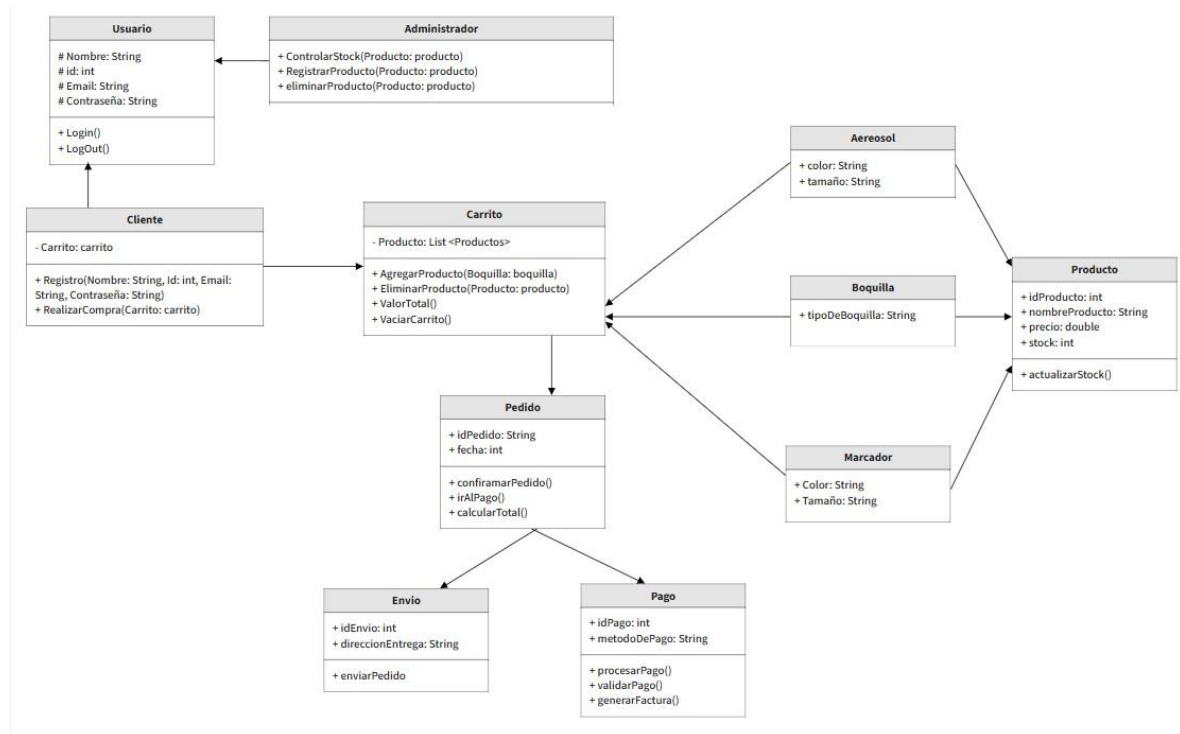
Las tiendas en línea son plataformas digitales creadas para vender productos o servicios a través de internet, permitiendo la interacción entre los usuarios y el sistema sin necesidad de un lugar físico. Estas plataformas suelen integrar elementos como la gestión de clientes, productos e inventario, lo que facilita la organización de la información y la automatización de los procesos comerciales. En el contexto del desarrollo de software, las tiendas en línea sirven como un ejemplo práctico para aplicar conceptos como la programación orientada a objetos y el modularidad dentro de un entorno funcional (Laudon & Laudon, 2016).

4.1. Funcionamiento de tiendas online

El funcionamiento de una tienda en línea se desarrolla a través de una serie de procesos organizados que permiten al usuario interactuar con el sistema para realizar una compra. En primer lugar, el cliente ingresa a un catálogo de productos donde puede consultar la información disponible y elegir los artículos que le interesan. Luego, estos productos se añaden a un carrito de compras, en el cual se gestionan los datos necesarios para continuar con la compra. Finalmente, el sistema se encarga de procesar el pago, verificar la información y confirmar el pedido, integrando distintos módulos como la gestión de usuarios, productos y transacciones para asegurar el correcto funcionamiento del sistema (Turban et al., 2015).

Diagramas UML

Diagrama de clases



Casos de uso

- Login() : Usuario.
- Logout() : Usuario.
- ControlarStock() : Administrador.
- RegistrarProducto() : Administrador.
- EliminarProducto() : Administrador.
- Registro() : Cliente.
- RealizarCompra() : Cliente.
- AgregarProducto() : Carrito.
- EliminarProducto() : Carrito.
- ValorTotal() : Carrito.

- VaciarCarrito() : Carrito.
- ActualizarStock() : Producto.
- ConfirmarPedido() : Pedido.
- irAlPago() : Pedido.
- CalcularTotal() : Pedido.
- ProcesarPago() : Pago.
- ValidarPago() : Pago.
- GenerarFactura() : Pago.
- EnviarPedido() : Envio.

Historias de casos de uso

1. Acceso al Sistema

Actores: Usuario

Descripción: El usuario entra a la plataforma y rellena el formulario con su correo y contraseña. El sistema comprueba si los datos coinciden con algún registro. Si todo está en orden, le da paso a la pantalla de inicio adaptada a su perfil (cliente o administrador). Si hay algún fallo, salta una alerta de credenciales incorrectas.

Atributos: Email, Contraseña.

Precondiciones: El usuario ya debe tener una cuenta creada.

Postcondiciones: Sesión iniciada y herramientas del rol habilitadas.

2. Salida del Sistema

Actores: Usuario

Descripción: Estando dentro de su cuenta, el usuario pulsa en "Cerrar sesión". En ese momento, el sistema corta la sesión activa, borra los tokens o datos temporales y lo manda de vuelta al inicio de la web.

Precondiciones: Haber iniciado sesión previamente.

Postcondiciones: Cierre de sesión efectivo; se requiere volver a loguearse para entrar.

3. Alta de Producto

Actores: Administrador

Descripción: Desde el panel de gestión, el administrador elige "Registrar producto".

Completa los campos de nombre, precio, stock y categoría (ya sea aerosol, boquilla o marcador). El sistema revisa que no falten datos, guarda el artículo y confirma que la operación fue exitosa.

Atributos: nombreProducto, precio, stock.

Precondiciones: El administrador debe estar autenticado.

Postcondiciones: El nuevo artículo aparece en la base de datos.

4. Baja de Producto

Actores: Administrador

Descripción: El administrador busca el producto en la lista, lo selecciona y confirma que quiere borrarlo. El sistema busca el registro y, tras verificar que existe, lo elimina definitivamente, avisando al usuario que el proceso terminó.

Precondiciones: Sesión de admin activa y que el producto esté listado.

Postcondiciones: El producto deja de existir en el sistema.

5. Gestión de Inventario

Actores: Administrador

Descripción: El administrador entra al inventario para revisar o ajustar las existencias de un producto concreto. Tras localizarlo, puede sobrescribir la cantidad de stock. El sistema valida el cambio y actualiza la cifra en el sistema.

Atributos: idProducto, stock.

Precondiciones: Estar logueado como administrador.

Postcondiciones: Stock actualizado correctamente.

6. Finalizar Compra

Actores: Cliente

Descripción: Con los artículos ya en el carrito, el cliente le da al botón de "Comprar". El sistema agrupa los productos en un pedido, suma los costos, lo manda a la pasarela de pago y deja listo el trámite para el envío.

Precondiciones: Sesión de cliente activa y productos cargados en el carrito.

Postcondiciones: Generación del pedido e inicio del pago.

7. Añadir al Carrito

Actores: Cliente

Descripción: Mientras navega por la tienda, el cliente elige un producto y lo añade a su cesta. El sistema chequea que haya unidades disponibles. Si hay stock, mete el producto al carrito y recalcula el importe total de la compra.

Atributos: idProducto.

Precondiciones: Usuario logueado; producto con existencias.

Postcondiciones: Producto guardado en el carrito y total actualizado.

8. Quitar del Carrito

Actores: Cliente

Descripción: El cliente revisa su carrito y decide sacar un artículo. El sistema lo retira de la lista y ajusta el precio total de la compra de inmediato.

Precondiciones: Sesión iniciada y el producto debe estar en la cesta.

Postcondiciones: Artículo eliminado y total corregido.

9. Limpiar Carrito

Actores: Cliente

Descripción: El cliente vacía toda la cesta de un solo clic. El sistema borra todos los artículos seleccionados y pone el contador de la cuenta a cero.

Precondiciones: Usuario autenticado.

Postcondiciones: Cesta vacía.

10. Validación de Pedido

Actores: Sistema / Cliente

Descripción: Cuando el cliente confirma que quiere cerrar la compra, el sistema hace un último chequeo de disponibilidad y prepara los detalles finales para pasar al cobro.

Precondiciones: Carrito con al menos un producto.

Postcondiciones: Pedido listo para pagar.

11. Gestión del Pago

Actores: Sistema

Descripción: El sistema procesa los datos del método de pago que el cliente eligió. Se conecta con la pasarela para aprobar o rechazar la transacción según el resultado del banco.

Precondiciones: Pedido confirmado previamente.

Postcondiciones: Pago registrado (éxito o fallo).

12. Emisión de Factura

Actores: Sistema

Descripción: En cuanto el pago se confirma, el sistema crea un comprobante o factura con el desglose de productos, precios individuales y el monto total abonado.

Postcondiciones: Factura generada y vinculada al pedido.

13. Despacho de pedido

Actores: Sistema

Descripción: Se toma la dirección de entrega facilitada por el cliente y se crea la orden de logística para que el paquete salga de almacén.

Atributos: dirección Entrega.

Postcondiciones: Pedido marcado en estado "Enviado" o "En proceso de envío".

Casos de uso

Diagrama 1:

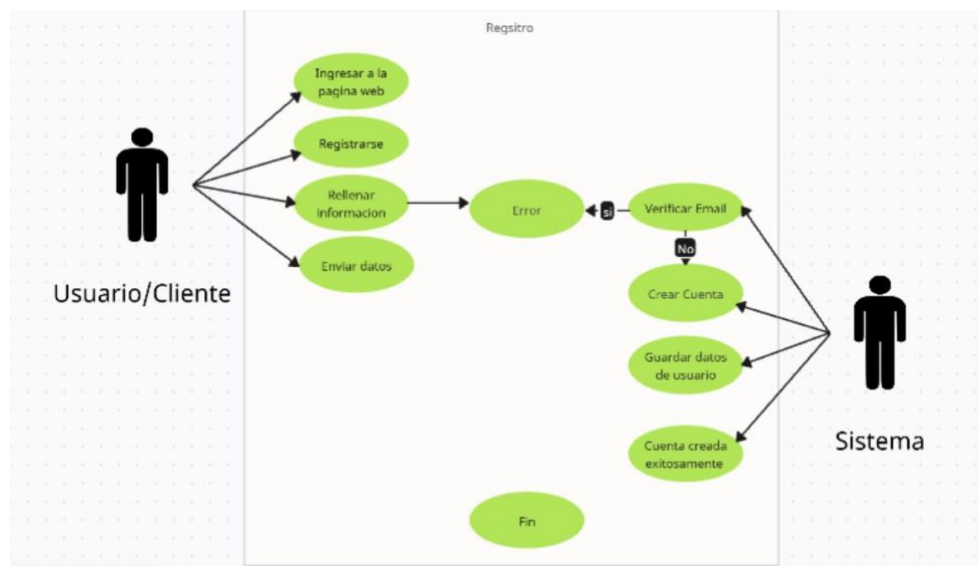


Diagrama 2:

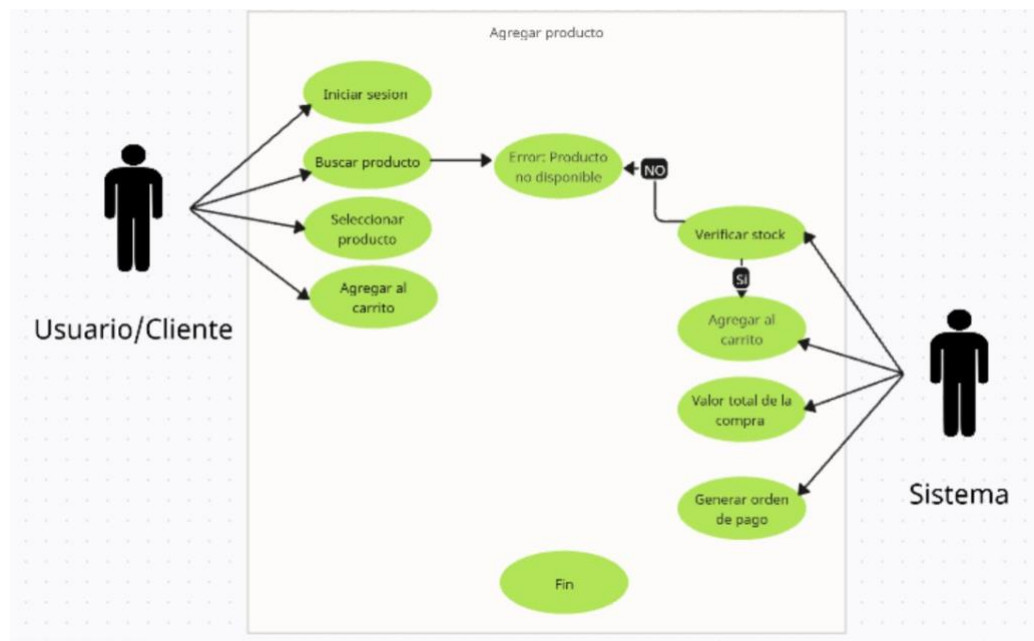


Diagrama 3:

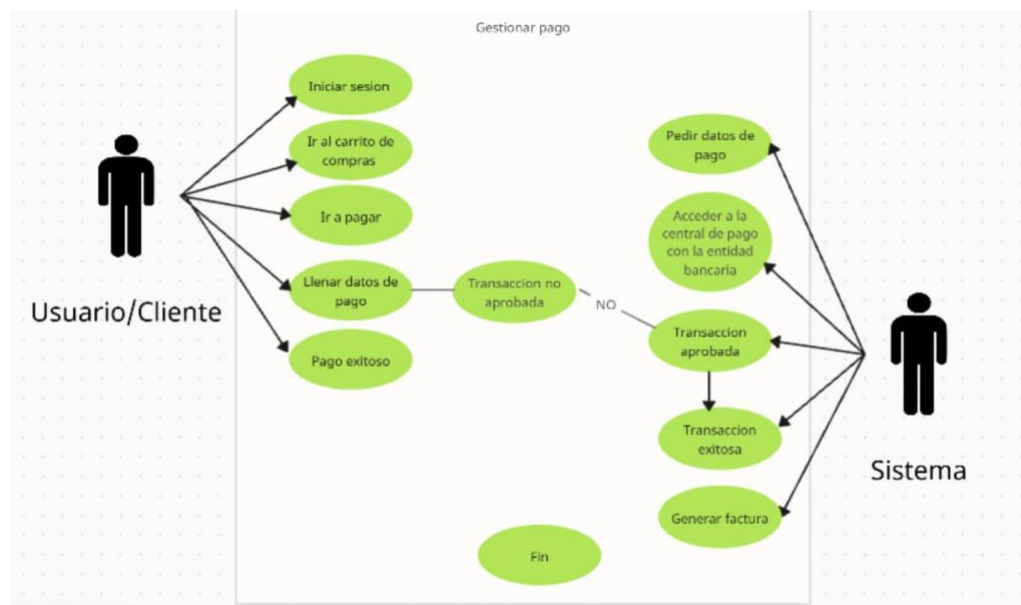
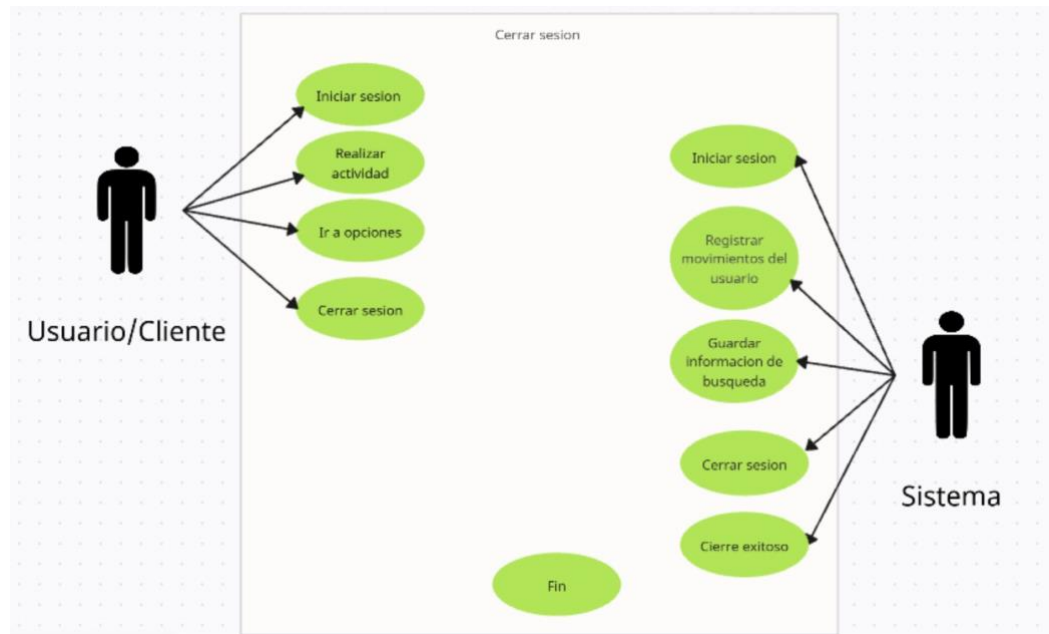


Diagrama 4:



Diagramas de actividades

Diagrama 1:

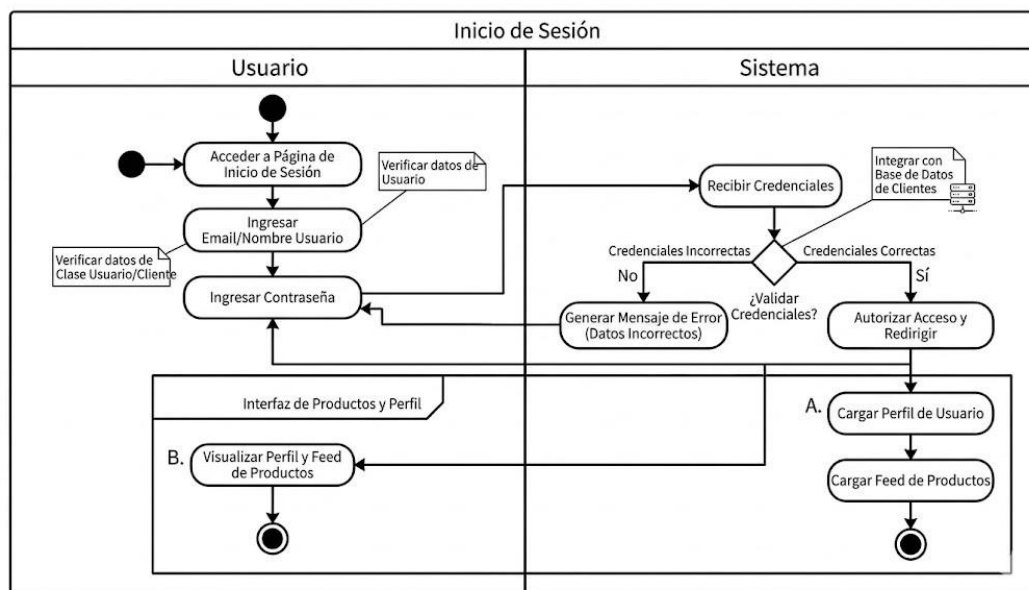


Diagrama 2:

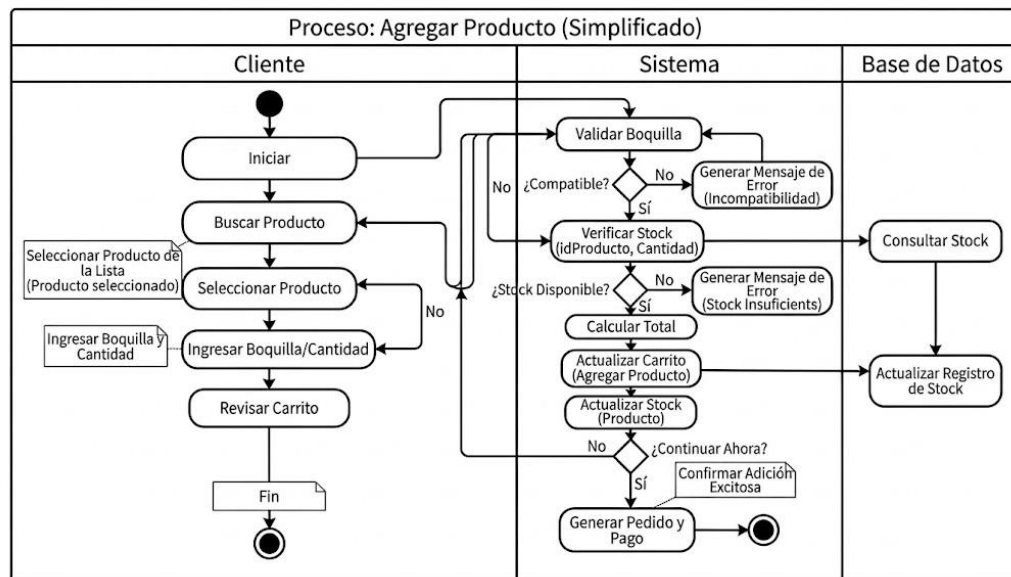


Diagrama 3:

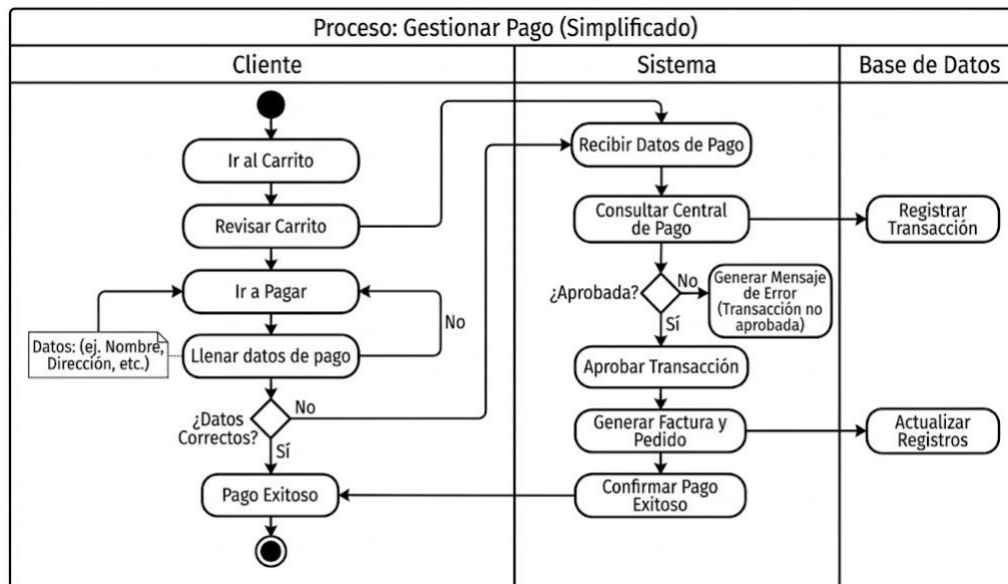
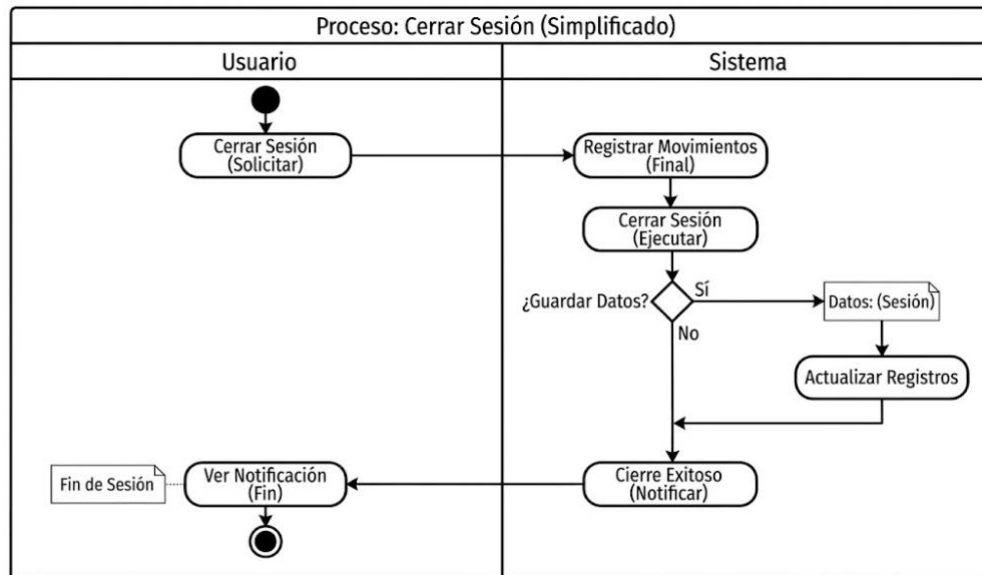
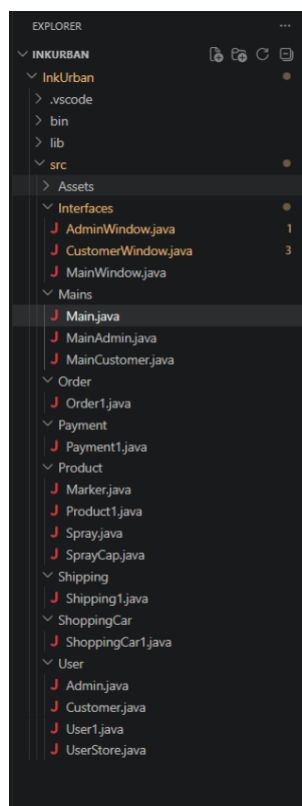


Diagrama 4:**Procedimiento:**

El código consta de clases de 18 clases en la cuales se trato de absorber la mayor cantidad de información del cliente con el fin de hacerlo lo mas parecido a una tienda oficial. En todas las clases se trabajaron los 4 pilares de POO y las GUI (Graphical User Interface).



En cada una de las clases se manejan excepciones para evitar que el usuario ingrese mal los datos.

Todas las clases tienen el mismo modelo que se muestra a continuación.

```
public void setName(String name) {
    if (name == null || name.trim().isEmpty()) {
        throw new IllegalArgumentException(s: "Este espacio no puede estar vacio");
    }
    this.name = name.trim();
}

public void setId(String id) {
    if (id == null || id.trim().isEmpty()) {
        throw new IllegalArgumentException(s: "El ID no puede estar vacio");
    }
    String cleanId = id.trim();
    if (cleanId.startsWith(prefix: "-")) {
        throw new IllegalArgumentException(s: "No se permiten numeros negativos");
    }
    if (!cleanId.matches(regex: "\\d+")) {
        throw new IllegalArgumentException(s: "El ID solo debe contener numeros");
    }
    this.id = Integer.parseInt(cleanId);
}

public void setMail(String mail) {
    if (mail == null || !mail.contains(s: "@") || !mail.contains(s: ".")) {
        throw new IllegalArgumentException("Correo invalido: \"" + mail + "\" (formato esperado: usuario@dominio.com)");
    }
    this.mail = mail.trim();
}

public void setPassword(String password) {
    if (password == null || password.trim().isEmpty()) {
        throw new IllegalArgumentException(s: "Este espacio no puede estar vacio");
    }
    if (password.length() < 8) {
        throw new IllegalArgumentException(s: "La contrasena debe tener minimo 8 caracteres");
    }
    this.password = password.trim();
}
```

```
public void setIdProduct(String idProduct) {
    if (idProduct == null || idProduct.trim().isEmpty()) {
        throw new IllegalArgumentException(s: "El id del producto no puede estar vacio");
    }
    String cleanId = idProduct.trim();
    if (cleanId.startsWith(prefix: "-")) {
        throw new IllegalArgumentException(s: "No se permiten numeros negativos");
    }
    if (!cleanId.matches(regex: "\\d+")) {
        throw new IllegalArgumentException(s: "El id del producto solo debe contener numeros");
    }
    this.idProduct = Integer.parseInt(cleanId);
}

public void setNameProduct(String nameProduct) {
    if (nameProduct == null || nameProduct.trim().isEmpty()) {
        throw new IllegalArgumentException(s: "Este espacio no puede estar vacio");
    }
    this.nameProduct = nameProduct.trim();
}

public void setPrice(String price) {
    if (price == null || price.trim().isEmpty()) {
        throw new IllegalArgumentException(s: "El precio no puede estar vacio");
    }
    String cleanPrice = price.trim();
    if (cleanPrice.startsWith(prefix: "-")) {
        throw new IllegalArgumentException(s: "No se permiten numeros negativos");
    }
    if (!cleanPrice.matches(regex: "\\d+(\\.\\d+)?")) {
        throw new IllegalArgumentException(s: "El precio solo puede contener numeros");
    }
    this.price = Double.parseDouble(cleanPrice);
}

public void setStock(String stock) {
    if (stock == null || stock.trim().isEmpty()) {
        this.stock = 0;
        return;
    }
    String cleanStock = stock.trim();
    if (cleanStock.startsWith(prefix: "-")) {
        throw new IllegalArgumentException(s: "No se permiten numeros negativos");
    }
    if (!cleanStock.matches(regex: "\\d+")) {
        throw new IllegalArgumentException(s: "El stock solo puede contener numeros");
    }
    this.stock = Integer.parseInt(cleanStock);
}
```

Se dividió el main en 3 clases, en la que se va a correr por completo el código, el main de administrador y el main del cliente, con finalidad de ocupar menos líneas de código en un solo main para evitar errores a largo plazo en la creación y ejecución del código.

En cada uno se recolectan los datos de usuario requeridos, y se ejecutan los métodos que se crearon para cada usuario del modo que se demuestra a continuación.

```
public void showMenu() {
    int option = 0;
    do {
        System.out.println(x: "Que desea hacer?");
        System.out.println(x: "1. Registrar producto");
        System.out.println(x: "2. Eliminar producto");
        System.out.println(x: "3. Controlar stock");
        System.out.println(x: "4. Cerrar sesion");
        try {
            option = Integer.parseInt(ky.nextLine());
            switch (option) {
                case 1:
                    addProducts();
                    break;
                case 2:
                    deleteProducts();
                    break;
                case 3:
                    controllStocks();
                    break;
                case 4:
                    admin.logout();
                    break;
                default:
                    System.out.println(x: "Opcion invalida");
                    break;
            }
        } catch (NumberFormatException e) {
            System.out.println(x: "Por favor ingrese un numero");
            option = 0;
        }
    } while (option != 4);
}
```

```
switch (op) {
    case 1:
        System.out.println(x: "Ingrese el color: ");
        String colorSpray = ky.nextLine();
        System.out.println(x: "Ingrese el tamaño: ");
        String sizeSpray = ky.nextLine();
        product = new Spray(idProduct, nameProduct, price, stock, colorSpray, sizeSpray);
        break;
    case 2:
        System.out.println(x: "Ingrese el color: ");
        String colorMarker = ky.nextLine();
        System.out.println(x: "Ingrese el tamaño: ");
        String sizeMarker = ky.nextLine();
        product = new Marker(idProduct, nameProduct, price, stock, colorMarker, sizeMarker);
        break;
    case 3:
        System.out.println(x: "Ingrese el tipo de tapa");
        String type = ky.nextLine();
        product = new SprayCap(idProduct, nameProduct, price, stock, type);
        break;
    default:
        System.out.println(x: "Opcion invalida");
        break;
}

if (product != null) {
    if (productIdExists(product.getIdProduct())) {
        throw new IllegalArgumentException(s: "Ya existe un producto con ese id");
    }
    products.add(product);
    admin.addProduct(product);
}
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}
}
```

Se emplearon clases de interfaces graficas que tienen el fin de crear lo que el usuario va a ver en pantalla con una estructura similar a la siguiente.

```

private JButton adminButton;
private JButton customerButton;
private JButton exitButton;
private JLabel logoLabel;
private JLabel subtitle;
private JPanel optionsPanel;

private List<Product1> products;
private UserStore userStore;

private final Color backgroundPurple = new Color(r: 37, g: 21, b: 54);
private final Color panelPurple = new Color(r: 70, g: 43, b: 101);
private final Color activePurple = new Color(r: 138, g: 82, b: 214);
private final Color darkPurple = new Color(r: 25, g: 16, b: 35);
private final Color white = new Color(r: 245, g: 242, b: 250);
private final Color pastelPurple = new Color(r: 241, g: 232, b: 250);

public MainWindow(List<Product1> products, UserStore userStore) {
    this.products = products;
    this.userStore = userStore;

    setTitle(title: "InkUrban");
    setSize(width: 560, height: 720);
    setLayout(new BorderLayout());
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setResizable(resizable: true);
    setExtendedState(JFrame.MAXIMIZED_BOTH);

    JPanel content = new GraffitiPanel();
    content.setLayout(mgmt: null);
    content.setBackground(backgroundPurple);
    add(content, BorderLayout.CENTER);

    ImageIcon logoIcon = cargarLogo();
    if (logoIcon != null) {
        setIconImage(logoIcon.getImage());
    }

    logoLabel = new JLabel(text: "", SwingConstants.CENTER);
    logoLabel.setOpaque(isOpaque: false);
    logoLabel.setBackground(new Color(r: 0, g: 0, b: 0, a: 0));
    logoLabel.setBounds(x: 25, y: 18, width: 510, height: 350);
    if (logoIcon != null) {
        Image logoSinFondoGris = limpiarFondoGris(logoIcon.getImage());
        logoLabel.setIcon(new ImageIcon(escalarImagenProporcional(logoSinFondoGris, maxWidth: 510, maxHeight: 350)));
    } else {
        logoLabel.setText(text: "INKURBAN");
        logoLabel.setFont(new Font(name: "Arial", Font.BOLD, size: 32));
        logoLabel.setForeground(white);
    }
    content.add(logoLabel);
}

```

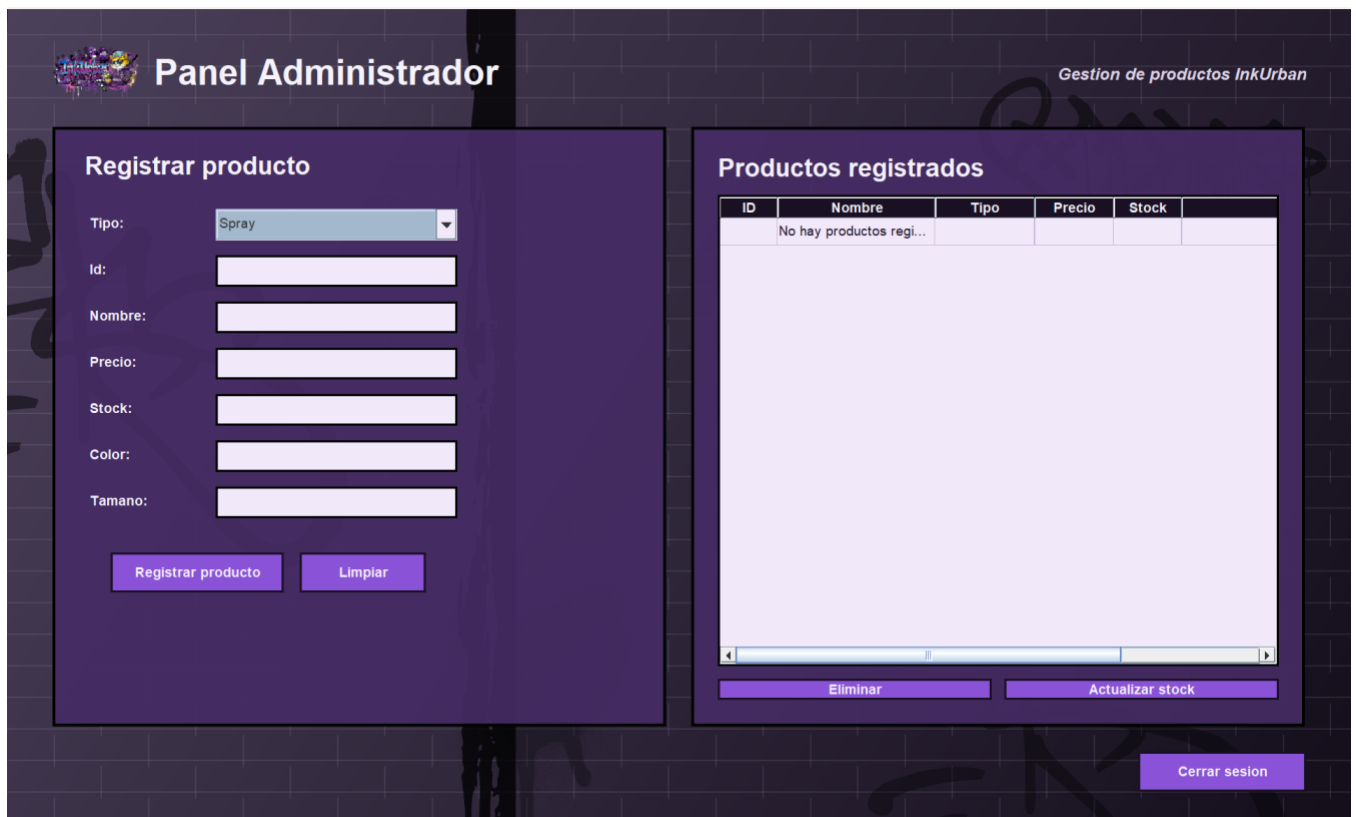
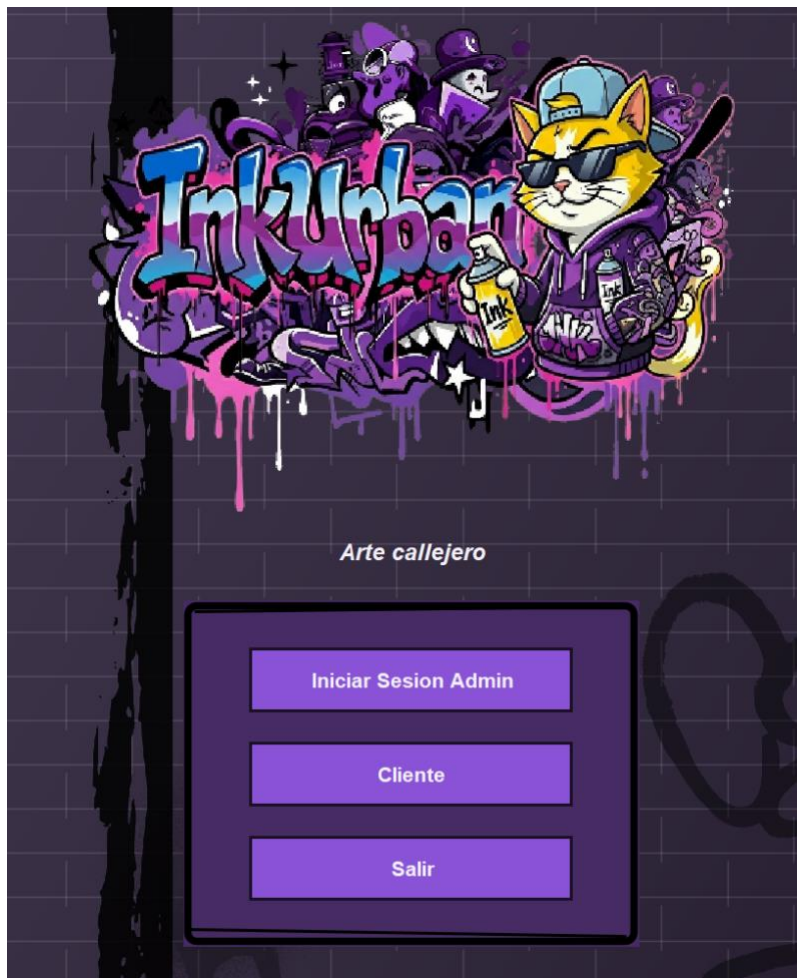
```

if ("Tarjeta de debito".equals(method) || "Tarjeta de credito".equals(method)) {
    extrasPanel.add(crearEtiquetaFormulario(text: "Nombre del titular:"));
    extrasPanel.add(crearCampoPago(name: "cardHolder"));
    extrasPanel.add(crearEtiquetaFormulario(text: "Numero de tarjeta:"));
    JTextField cardNumber = crearCampoPago(name: "cardNumber");
    cardNumber.setToolTipText(text: "16 digitos");
    extrasPanel.add(cardNumber);
    extrasPanel.add(crearEtiquetaFormulario(text: "Fecha de vencimiento (MM/AA):"));
    JTextField expirationDate = crearCampoPagoFecha(name: "expirationDate");
    expirationDate.setToolTipText(text: "Formato MM/AA");
    extrasPanel.add(expirationDate);
    extrasPanel.add(crearEtiquetaFormulario(text: "Codigo de seguridad:"));
    extrasPanel.add(crearCampoClavePago(name: "securityCode"));
} else if ("Transferencia".equals(method)) {
    extrasPanel.add(crearEtiquetaFormulario(text: "Banco:"));
    extrasPanel.add(crearCampoPago(name: "bank"));
    extrasPanel.add(crearEtiquetaFormulario(text: "Numero de cuenta o telefono:"));
    JTextField account = crearCampoPagoNumerico(name: "account", maxCharacters: 20);
    account.setToolTipText(text: "Solo numeros");
    extrasPanel.add(account);
    extrasPanel.add(crearEtiquetaFormulario(text: "Referencia de transferencia:"));
    extrasPanel.add(crearCampoPago(name: "reference"));
} else {
    JTextArea nota = new JTextArea(text: "Pago contra entrega. Ten el valor completo listo al recibir tu pedido.");
    nota.setEditable(b: false);
    nota.setLineWrap(wrap: true);
    nota.setWrapStyleWord(word: true);
    nota.setOpaque(isOpaque: false);
    nota.setFont(new Font(name: "Arial", Font.BOLD, size: 14));
    nota.setForeground(pastelPurple);
    extrasPanel.add(nota);
}

extrasPanel.revalidate();
extrasPanel.repaint();
}

```

Todo termina con la finalidad de que el cliente o administrador vea lo siguiente.





Conclusión

El desarrollo de este proyecto permitió aplicar de forma concreta los conocimientos trabajados durante el semestre, especialmente los cuatro pilares de la programación orientada a objetos, dentro de un contexto que simula una situación real de comercio electrónico. Diseñar el sistema de UrbanInk como una tienda en línea de productos para grafiti y muralismo fue una forma práctica de entender cómo se organizan y relacionan los distintos componentes de un software, desde la gestión de usuarios y productos hasta los procesos de pago y envío.

La aplicación del encapsulamiento, la herencia, el polimorfismo y la abstracción no quedó solo en la teoría, sino que se vio reflejada directamente en la estructura del sistema, en cómo se definieron las clases, sus relaciones y sus responsabilidades. Esto permitió construir un modelo más ordenado, donde cada parte cumple una función específica sin interferir con las demás.

Los diagramas UML fueron una herramienta clave durante todo el proceso, ya que permitieron visualizar el sistema antes de pensar en su implementación. Los diagramas de clases, casos de uso y actividades ayudaron a tener una visión clara de lo que el sistema debe hacer y cómo debe hacerlo, mientras que las historias de usuario acercaron el diseño a las necesidades reales de quien usaría la plataforma.

Finalmente, este proyecto dejó una base sólida para seguir desarrollando habilidades en el análisis y diseño de software. Más allá del resultado académico, el proceso en sí fue valioso porque obligó a pensar de forma estructurada, a tomar decisiones de diseño y a entender que construir un sistema bien organizado desde el principio facilita todo lo que viene después.

REFERENCIAS

- Booch, G. (2007). *Object-Oriented Analysis and Design with Applications* (3rd ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Sommerville, I. (2011). *Software Engineering* (9th ed.). Pearson.
- Schildt, H. (2017). *Java: The Complete Reference* (10th ed.). McGraw-Hill.
- Laudon, K. C., & Laudon, J. P. (2016). *Management Information Systems* (14th ed.). Pearson.

- Turban, E., King, D., Lee, J., Liang, T. P., & Turban, D. (2015). *Electronic Commerce: A Managerial and Social Networks Perspective*. Springer.